

A Method for Stochastic Modeling the Software Development Process in Constrained Resource Environments

Fernando Machado Lima Ferreira, Eber Assis Schmitz, Antonio Juarez Alencar,
Fabio Protti and Carlos Henrique Alves

Abstract—Process Improvement requires a formal process definition and associated measures of performance. Processes can then be diagnosed in order to formulate hypotheses on where changes should be made. Subsequently, new processes can be conceived, and developers can check whether the desired improvements have actually been achieved. Simulation is a powerful low-cost tool for diagnosis and test of several improvement alternatives, prior to field tests.

This paper presents a UML-based method to obtain the probability distribution of the execution time of a large variety of business processes, including software development. Such a method, which is based on Monte Carlo simulation, allows for the identification of factors that most strongly influence process execution time, favoring changes that increase process efficiency with considerable impact on the deployment of business tactics and strategies.

I. THE PROBLEM

Adopting a process view is a revolutionary change on how the work is performed in the organizational environment, contrasting with the emphasis on products. A process is a set of structured and measurable activities that yields a product or service to a determined client or market. A process is also an ordering of activities in time and space, with start and end, and inputs and outputs clearly defined, in other terms, an action-oriented structure [1].

Software development processes (SDP) are the business processes of Software Companies. Software development process models are created to attend many objectives: understanding the key mechanisms of the existing business, to create information systems to support the business, to act as a basis for improving the business, to show the structure of an innovated business, to experiment with a new concept and identify outsourcing opportunities [2].

Since the publication of Royce on the management of software development in [3], software development processes have been the subject of an extensive number of studies. Also, software process improvement model has been defined in the CMMI [4] which provides a guide for the continuous improvement of the development processes.

In a world of increasingly shorter time to market, where companies are required to innovate constantly, project cycle time has become one of the most critical software development parameters. If one cannot deliver the software that is required in the allowed time frame, someone else will

[5]. Nevertheless, every software development improvement project starts with a diagnostic phase, proceeds to an evaluation of possible alternatives, which includes the selection of the best course of action to be taken, and concludes with the presentation of the proposed solution [6].

Software process improvement entails substantial risk as it is, usually, very difficult to predict the outcome of changes before they are put into practice, and it is evident that the failure rates of software development process change related project are high. So, it seems reasonable to claim that simulation could offer a great potential in modeling and analyzing SDP, and therefore reduce the risks associated with process change. Simulation models can provide a graphical display of process models that can be interactively edited and animated to show process dynamics. Most important, such models enable conducting of "what if" analysis that can be used to evaluate the consequences of changes before they are implemented.

Over the years simulation has revealed itself to be an important tool to speed up software development improvement both in the diagnostic and the alternative evaluation phases. In the former, the simulation model is fed with data from the current software development process, whose analysis indicate the steps that require attention. In the latter, a proposed new model of the process is fed with estimated parameters. The analysis of the data collected during the simulation of the new model reveals the cycle time reduction achieved by modifying the current process [7].

SDPs are a form of stochastic process. The implication is that the process metrics will be random variables, in particular duration of a process, will be described by its Probability Distribution Function (PDF). This paper presents a new method for obtaining the PDF of the duration of a software processes. First a UML-based process model is created using Activity Diagrams, from this informal model a formal model is generated and simulated. The simulation results are then used to obtain an approximation to the PDF of the process duration.

This paper introduces a method for obtaining the probability distribution function (PDF) of the duration of a software development process. The method, which is based upon stochastic modeling, Monte Carlo simulation and the Unified Modeling Language (UML), allows managers to foresee more easily and quickly the consequences of changes in the software development process, even when resources are constrained and activities are carried out in finite loops.

This paper is organized as follows: section 2 describes the

Fernando M. L. Ferreira is a student with the Informatics Graduate Program at Federal University of Rio de Janeiro. Professors Eber A. Schmitz, an affiliate of IEEE, and Fabio Protti are Mr. Ferreras's faculty supervisors. Dr. Antonio J. Alencar is a researcher with the Federal University of Rio de Janeiro.

process modeling using UML Activity Diagrams, section 3 presents a method for estimating the duration of processes. Section 4 describes an example. Section 5 presents a discussion of our findings. Finally section 6 shows the conclusions.

II. RELEVANT LITERATURE

The inadequacy of the use of averages in project scheduling is discussed in detail by Elmaghra [8]. Nelson [9] provides a comprehensive introduction to stochastic modeling, simulation and analysis. The state of the art of methods to deal with project scheduling under uncertainty is discussed by Herroelen and Leus [10]. This includes software development improvements projects. It is important to mention that none of the methods mentioned in [10] allows for loops of activities. The software development improvement model has been defined in the Capability Maturity Model Integration (CMMI), which provides a guide for the continuous improvement of the development process [4].

An activity network (AN) is a graph where the nodes represent activities and edges represent precedence among activities. The most common precedence relation between two activities is the finish-to-start (FS), which restricts the start of execution of one activity to the termination of another. There are two main types of ANs: deterministic and non-deterministic. The deterministic activity network (DAN) is represented by a graph whose topology remains fixed as the activities are being executed. Examples of DANs include the widely known CPM and PERT networks.

On the other hand, an activity network contains decision and repetition nodes which allow for constructions that, not only restrict the order of the activities, but also whether they are going to be executed or not and the number of times it can be executed. Other examples of activity networks in this class are Markov chains, Generalized Activity Networks (GAN) [11] and networks used in the Graphic Evaluation and Review Technique (GERT) [12] and Controlled Alternative Activity Networks (CAAN) [13].

The UML owes its origins to the work of Booch, Rumbaugh and Jacobson in the 1990's on the unification of their respective object-oriented system specification methodologies. Nowadays, the UML is maintained and further developed by the Object Management Group (www.uml.org). Although the UML is a widely recognized and used standard, it has been criticized for imprecise semantics leading to subjective interpretation and, consequently, difficulties in formal business process modeling [14]. Hence a restricted form of UML is used throughout this paper.

III. PROCESS MODELING USING UML ACTIVITY DIAGRAMS

In the extremely competitive markets where software is developed these days, the Unified Modeling Language (UML) is becoming the *de facto* standard for describing software specification, structure, behavior and architecture [15]. The UML is also being used as a language to describe business models. The UML comprises several diagram types,

each one devised to represent a different view of the information system. The UML Activity Diagram (AD) is a simple yet effective modeling notation. Its main uses range from describing a simple activity to a complex organizational workflow. The AD emphasizes the flow of control between activities. Also, the resources required and the products generated by each activity have an explicit representation. In summary, the AD is able to describe the dynamics of software development processes. In this respect, the AD is similar to the flow graph and the EPC (Event-driven Process Chain) [16].

Throughout this paper a subset of the UML AD symbols is used to describe process models. This subset comprises: process delimiter (Start/End), activity, transition, branch (Decision/Join), synchronism bar (Fork/Join). Figure 1 introduces these symbols.

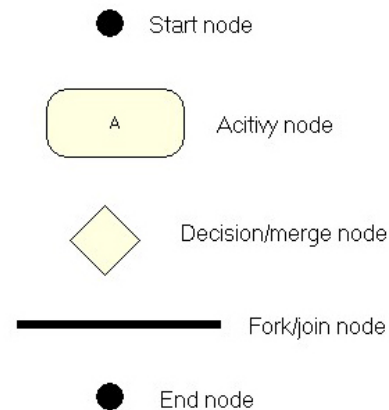


Fig. 1. UML Process elements.

IV. THE METHOD

In formal terms, to obtain the PDF of a software development process in a constrained resource environment one should

- 1) Create a well formed process model (WFP-model) using the UML AD;
- 2) Simulate the WFP by repeating the following activities:
 - a) Generate randomly a possible scenario (which is expressed as an RC-DAN) and
 - b) Generate a schedule of its activities using a predefined business priority rule.

The set of schedules generated by simulation provides an estimate of the PDF of the process execution time. The number of times Step 3 is executed can be calculated with the support of the Kolmogorov-Smirnov (KS) limit theorem for empirical distributions [17]. The paper proceeds by describing each one of the steps above.

A. Well Formed UML Process models

Before discussing the execution simulation a WFP some definitions are needed.

Definition 1: A well defined activity is an activity in which the PDF of its duration and the resources that it requires to be executed are specified $\square$

Definition 2: A well defined decision node is a decision node in which the discrete probability distribution associated to each branch node is specified $\square$

Definition 3: A well defined loop node is a loop node where the probability distribution of the number of times it is executed is specified $\square$

Definition 4: A well defined activity is a well formed process or WFP. If $WFP_1, WFP_2, \dots, WFP_n$ are well formed processes then the network shown in Figure 2 is a WFP $\square$

Definition 5: If $WFP_1, WFP_2, \dots, WFP_n$ are well formed processes, then the network shown in Figure 2 is a WFP $\square$

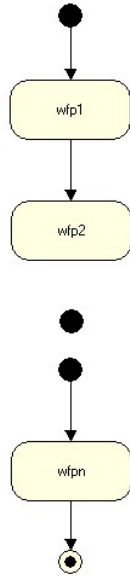


Fig. 2. Series combination of WFPs.

Definition 6: If $WFP_1, WFP_2, \dots, WFP_n$ are well formed processes and D1 is a well defined decision node, then the network in Figure 3 is a WFP $\square$

Definition 7: If $WFP_1, WFP_2, \dots, WFP_n$ are well formed processes and D1 is a well formed loop node, then the network in Figure 4 is a WFP $\square$

Definition 8: A resource constrained deterministic activity network (RC-DAN) is an instance of a WFP, where the outcome of all loop and single decision nodes have already been determined $\square$

B. Simulating the execution of a WFP

A WFP contains decision and repetition nodes which allow for constructions that, not only restrict the order of the activities, but also whether they are going to be executed or not and the number of times they can be executed.

WFPs restrain the execution of an activity to the availability of the required amount of resources. The total amount of resources required for the execution of the process is defined by the resource vector: $R[1..q]$, where $R[i]$ is the total availability of resource of type i .

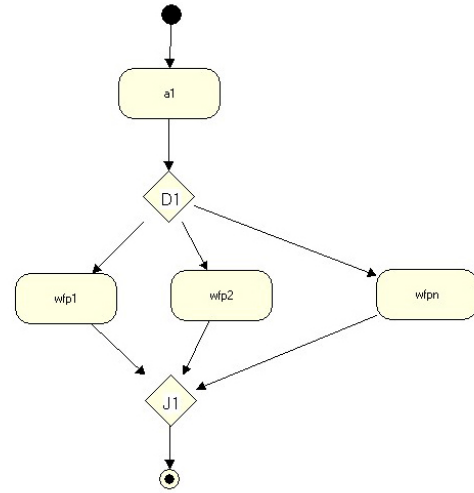


Fig. 3. Decision combination of WFPs.

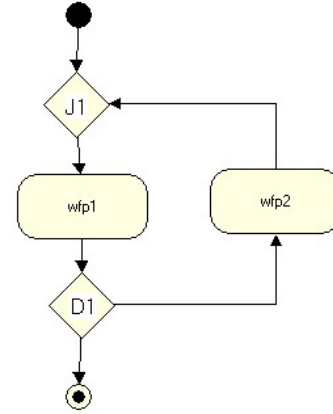


Fig. 4. Loop combination of WFPs.

The execution of a WFP requires the definition of a priority rule H , which is a function that associates to each activity a_i a unique number $H(a_i)$, such that if $H(a_i) > H(a_j)$ then a_i has a preference to be executed before a_j .

The problem of estimating the duration of a software development process can be seen as a variant of the heuristic solution for the Stochastic Resource Constrained Project Scheduling Problem (Stochastic RCPSP) where the objective function is the minimization of total project duration (makespan) [18], [19].

The heuristic approach to the solution of the RCPSP minimum makespan consists of the use of a decision policy when deciding the priority of execution of a set of tasks competing for the same resources. Examples of decision policies are: LST (least start time), GRU (great resource utilization) and FCFS (first-come first-server). See [20] for an empirical study of the performance of these policies on makespan minimization.

This paper proposes the use of the same heuristic method for the solution of the minimization of makespan in deterministic project networks to obtain the PDF of the execution time

of a SDP. To use this method one must take two elements into consideration: (1) the business policy for resource allocation can be converted into a priority rule (heuristic) and (2) the SDP can be modeled as a WFP.

These two assumptions allow the determination of the execution time of software development processes described by WFPs with the same algorithm used in the solution of RCPSPs. The algorithm is used in conjunction with the Monte Carlo (MC) method to obtain an approximation of the distribution of a process execution time.

The MC method is basically a sampling procedure over the universe of all possible executions of the process. Each sample is called a scenario. Hence a scenario is an instance of the WFP where every activity has a known duration and the outcoming branch of all decision nodes are defined. The application of the decision policy to a given instance allows us to determine, in a unique manner, the total execution time of the process instance.

The simulation method presented is based on the repetitive execution of two main algorithms: GenerateDAN and ScheduleDAN. The number of repetitions is defined by a constant NumScenarios.

SimulateWFP (WFP, R, H, NumScenarios)

CountScenarios = 0;

repeat

dan = GenerateDAN(WFP);

ScheduleDAN(dan, R, H);

inc(CountScenarios);

Until CountScenarios > NumScenarios

C. The algorithm GenerateDAN

This algorithm is responsible for obtaining a deterministic instance of the WFP by selecting one of its possible execution paths. This procedure first transforms the WFP into one of the possible RC-DANs, by replacing all decision nodes with a single branch, representing the realization of the decision. In a subsequent step, a constant duration value is sampled for each activity from the its duration PDF. The definition of a RC-DAN is exactly the same of a WFP, except for the fact that all decision nodes have been instantiated.

GenerateDAN basically has to deal with two types of nodes, i.e. decision and loop. Initially consider the graph introduced in Figure 3. In this case one of the branching WFPs emanating from the decision node D1 is selected and replaces the whole subgraph connecting D1 to J1. Now consider the graph introduced in Figure 4. In this case replace graph connecting J1 to L1 is replaced by WPF1 followed by a number of sequences WFP1 WFP2.

Figure 5 shows two examples of transformation of a WFP into a RC-DAN. In the figure to the left, the WFP contains a decision node D1 and two mutually exclusive activities a1 and a2 followed by the node J1. In order to get its instantiated DAN, one must choose the branch to be taken. Suppose that the Y branch is selected, then both D1 and J1 are removed and yielding the DAN that contains one instance of the activity a1 called a1[1].

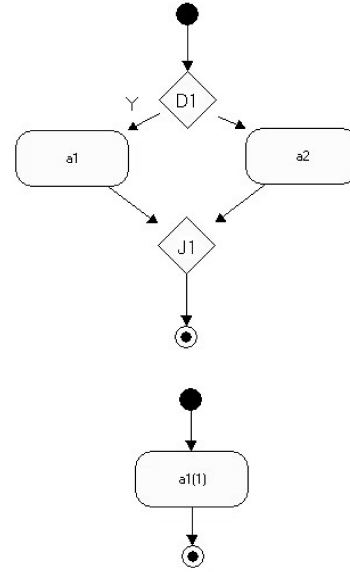


Fig. 5. WFP into a DAN

In the Figure 6, the WFP contains a loop. Suppose that loop L1 is executed just once, then both J1 and L1 are removed and one gets the DAN with a1[1] followed by the sequence a2[1] a1[2].

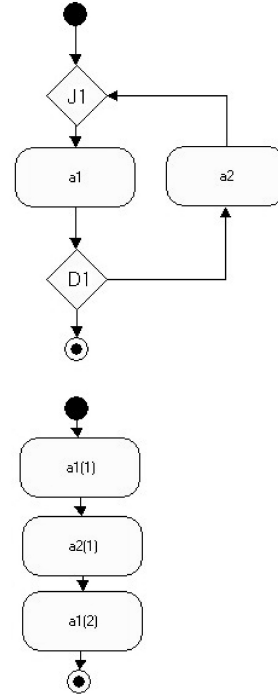


Fig. 6. WFP into a DAN

D. The algorithm ScheduleDAN

The second of these algorithms is the standard RCPS parallel algorithm described by [20] to the application of a heuristic for the solution of the deterministic RCPSP (RC-DAN) generated by the GenerateDAN Algorithm. The

application of this algorithm evaluates the process execution duration, for the selected scenario. This set of values, can be used to obtain both the PDF and the Cumulative Distribution Function (CDF) of the process execution duration.

V. PRELIMINARY RESULTS

To illustrate the method consider the estimation of the execution time of a software development process which is based on the Incremental Development Method [21]. A simplified process, including only technical activities, is shown in Figure 7

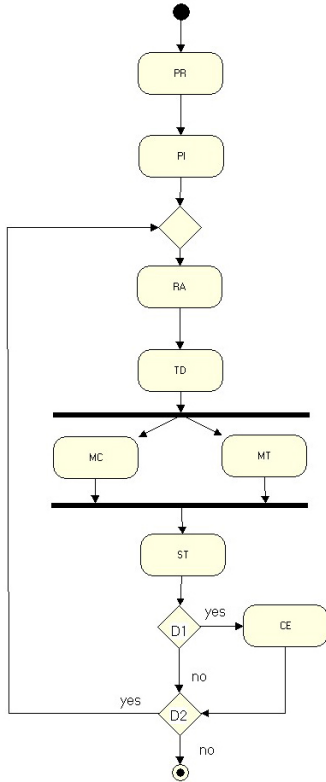


Fig. 7. Software process model.

The process starts with a preliminary requirements analysis (PR), followed by the package identification (PI) whose objective is partitioning the system into packages, which are implemented one at a time. Each package is then subjected to a thorough detailed analysis of requirements (RA), followed by a technical design phase (TD). The design phase, produces a set of software modules which are coded (MC) concurrently with their testing (MT). In the next phase the set of modules is tested as a whole (ST), and there is a finite probability that they will fail, requiring some rework to the package error correction (EC).

The decision node D1 has two possible transitions (yes or no) with equal probability of being used. Moreover, the decision node D2 is set to loop four times. Hence the project will be divided into four packages. The resources available are one manager (M) one designer (D) and two programmers

(P). Table I shows the process activities, the resources required to its execution and the parameters (Minimum, Most likely and Maximum, in hours) of the triangular PDF of its duration.

TABLE I
ACTIVITIES, RESOURCES AND DURATION.

Activity	Resource	Min	Most Likely	Max
PR	1 M, 1 D	88.0	110.0	132.0
PI	1 M, 1 D	138.0	172.5	143.7
RA	1 M, 1 D	27.6	34.5	41.4
TD	1 M, 1 D	83.6	104.5	125.4
MC	1 M, 2 P	65.06	81.3	97.6
MT	1 M, 1 P	23.6	29.5	35.4
ST	1 M, 2 P	34.4	43.0	51.6
EC	1 M, 2 P	17.06	21.33	25.6

According to the KS limit theorem [17], to obtain a maximum error of $D_{n,\alpha}$ of 2% with 95% of confidence level it is necessary to generate at least 4,649 observations. Assuming a 80/20 threshold, where 80% of the scenarios are covered, the process execution time is estimated to be $1,810 \pm 36.2$ hours with 95% confidence level and 2% of maximum error. Considering a working day of 8 hours, the process execution time is estimated to be 227 ± 5 calendar days.



Fig. 8. Frequency and Cumulative Functions

VI. DISCUSSION

Below we answer some of the key questions regarding the use of the method and techniques described in this article to estimate the duration of a software development process.

How does the method proposed in this article favor the improvement of process execution time?

The method is built upon a mathematical model that more realistically reflects the complex and dynamic environment in which the software development process takes place, where activity execution time, availability of resources and the execution of some activities are uncertain.

By making it easier for developers to model, analyze and change processes in a more realistic way, the method proposed in this article favors the continuous shortening of time

spent on the development of software and in the improvement of process execution time in many other industries.

How does one choose among several improvement alternatives?

Because, in general, each conceived process improvement alternative has to be simulated and each one yield a different probability distribution function, it is likely that software developers end up in a situation where they have to choose one among several possible PDFs, where each one has its own mean and standard deviation

Although, this problem falls into the realm of decision theory, if one establishes a maximum variation from the mean that is acceptable, i.e. a maximum risk that one is willing to take that the processes actually finishes within a given interval, the obvious choice among the alternatives is the improvement that has the smallest mean.

What are the implication of such method to a software company tactics and strategies?

If a company has processes that are efficient and yield some meaningful savings is cost, it may use this savings either to open new markets that could not be reached due to cost restriction or to conquer additional market share in the environment in which it does business. Alternatively, such a company can pass on some of the saving it earned as a result of process improvement to some of its clients, with the view of making the process of those clients more efficient and as a means of establishing a stronger relationship with those who are important to business.

Therefore, by favoring process improvement, the method present in this article favors the conception and deployment of many business tactics and strategies that could not be done otherwise.

VII. CONCLUSIONS

Cycle time is one of the most important business parameters today. All business improvement projects start with a diagnosis phase, proceed to an evaluation of possible alternatives and conclude with the presentation of the proposed solution.

Simulation is an important tool both in the diagnostic and in the alternative evaluation phases. In the diagnostic phase, the model is fed with actual data from the business processes. The simulation results can be used to diagnose the process steps, which require attention.

In the alternatives evaluation phase, the proposed new model of the process is fed with the new estimated parameters. An analysis of the simulation data will reveal the cycle time reduction achieved by the new process.

This paper introduced a method for obtaining the probability distribution of the duration of a software development process. The method starts with a description of the process using the UML notation according to some formation rules (WFP). The simulation is based on the Monte Carlo approach. It starts by sampling one possible instance of WFP, which is then expressed as a Resource Constrained Deterministic Network (RC-DAN). The execution time of the RC-DAN is obtained by scheduling its activities according to

a predefined business policy. Defining the number of samples allows us to estimate the cumulative distribution time of the process, within a given confidence interval, with a required confidence level.

REFERENCES

- [1] M. Hammer and J. Champy, *Reengenharia: revolucionando a empresa*, 5th ed. Rio de Janeiro, RJ: Campus, 1994.
- [2] H. E. Eriksson and M. Penker, *Business Modeling With UML: Business Patterns at Work*, 1st ed. USA: Wiley, 2000.
- [3] W. W. Royce, "Managing the development of large software systems: Concepts and techniques," *Proceedings WesCon*, August 1970.
- [4] M. B. Chrissis, M. Konrad, and S. Shrum, *CMMI Guidelines for Process Integration and Product Improvement*. Addison Wesley, 2004.
- [5] R. Baskerville and J. Pries-Heje, "Short cycle time systems development," *Information Systems Journal*, vol. 14, no. 3, pp. 237–264, 2004.
- [6] B. Hansena, J. Roseb, and G. Tjørnehøj, "Prescription, description, reflection: the shape of the software process improvement field," *International Journal of Information Management*, vol. 24, no. 6, pp. 457–472, 2004.
- [7] M. Ruiz, I. Ramos, and M. Toro, *New Trends in Software Process Modelling*. World Scientific, 2006, ch. 2: Software Process Dynamics: Modeling, Simulation and Improvement, pp. 21–56.
- [8] S. E. Elmaghra, "On the fallacy of averages in project risk management," *European Journal of Operational Research*, vol. 165, no. 2, pp. 307–313, September 2005.
- [9] B. L. Nelson, *Stochastic Modeling: Analysis & Simulation*. Dover Publications, February 2003.
- [10] W. Herroelen and R. Leus, "Project scheduling under uncertainty: Survey and research potentials," *European Journal of Operational Research*, vol. 165, no. 2, pp. 289–306, September 2005.
- [11] S. E. Elmaghraby, *Activity networks : project planning and control by network models*. New York: Wiley, 1997.
- [12] L. J. Moore and E. R. Clayton, *GERT Modeling and Simulation: Fundamentals and Applications*. Melbourne: Krieger Publishing Co., 1976.
- [13] D. Golenko-Ginzburg, A. Gonik, and Z. Laslo, "Resource constrained scheduling simulation model for alternative stochastic network projects," *Mathematics and Computers in Simulation*, vol. 63, pp. 105–117, 2003.
- [14] H. Eshuis and R. J. Wieringa, "A formal semantics for UML activity diagrams - formalising workflow models," University of Twente, Enschede, The Netherlands, Tech. Rep., 2001.
- [15] *UML Resource Page*, Object Management group, 2006, access at: March 23rd, 2006. [Online]. Available: <http://www.uml.org/>
- [16] W. M. P. Aalst, "Formalization and verification of event-driven process chains," *Information and Software Technology*, vol. 41, no. 10, pp. 639–650, 1999.
- [17] W. Feller, "On the Kolmogorov-Smirnov limit theorems for empirical distributions," *The Annals of Mathematical Statistics*, vol. 19, no. 2, pp. 177–189, June 1948.
- [18] E. L. Demeulemeester and W. S. Herroelen, *Stochastic project scheduling*, 1st ed. USA: Springer, January 2002.
- [19] B. Yang, J. Geunes, and W. O'Brien, "Resource constrained project scheduling: Past work and new directions," University of Florida, Enschede, The Netherlands, Tech. Rep., April 2001.
- [20] R. A. V. Olaguibel and J. M. T. Goerlich, *Heuristic Algorithms for Resource-Constrained Project Scheduling: A Review and Empirical Analysis*, R. Stowinski and J. Weglarz, Eds. Elsevier Science Publishers B. V., 1989.
- [21] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 6th ed. McGraw-Hill, 2004.